VERIQTA

SHELL SCRIPTING FROM SCRATCH

A Practical, Hands-On Guide to Writing, Automating, and Managing Tasks with Bash

```
$ cat deploy.sh
#!/bin/bash
echo "Deploying..."
docker compose up -d
echo "Done!"
$
```



- Automate Tasks
- Process Files
- Work with Data
- Schedule Jobs
- Build DevOps Tools



FOR BEGINNERS, JUNIOR ENGINEERS, AND DEVOPS PRACTITIONERS

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

Shell Scripting Fundamentals and Execution Model

What Is a Shell?

- A shell is a command-line interface (CLI) that lets you interact with the Linux system.
- It takes your commands, interprets them, and tells the OS what to do.
- Examples: Bash, Zsh, Ksh, Fish.

Bash vs Other Shells

Shell	Details	Common Use
Bash	Default on most Linux systems.	Scripting, system administration.
Zsh	Feature-rich, user friendly.	Developer productivity.
Ksh	Stable, enterprise friendly.	Enterprise and legacy systems.
Fish	Modern and interactive.	Easy, interactive use (not common for production scripts).

Scripts vs Interactive Commands

- **Interactive command:** You type and run a command directly in the shell (e.g., ls, pwd).
- **Script:** A file that contains multiple commands, saved and executed together (e.g., deploy.sh).

Shebang (#!)

- A shebang tells Linux which shell interpreter to use for the script.
- It must be the first line of the script.
- Common example:

```
#!/bin/bash
```

Execution Permissions

- A script must be executable before you can run it.
- Make it executable with:

```
chmod +x script.sh
```

- Verify the permission:

```
ls -l script.sh
```

How Linux Executes a Shell Script



1. You run the script (e.g., ./script.sh)
2. Linux reads the shebang and loads the interpreter (e.g., /bin/bash)
3. The shell executes the commands in the script line by line.

Key Takeaways

- ✅ A shell interprets and runs commands.
- ✅ Bash is the most common shell for scripting.
- ✅ A shell script is a saved set of commands.
- ✅ Use a shebang to specify the interpreter.
- ✅ Use chmod +x to make a script executable.

Junior Engineer Tip



To avoid surprises, always add a shebang, set the correct permissions, and test your script with **./script.sh**.

Writing and Running Your First Bash Script

What You'll Learn

- Create a simple Bash script.
- Understand the script structure.
- Make it executable and run it.
- Use exit codes and basic debugging.

Script Structure

hello.sh

```
#!/usr/bin/env bash
# A simple Bash script
echo "Hello, DevOps Engineer!"
exit 0
```

The Shebang: #!/usr/bin/env bash

- The first line of a script.
- Tells Linux which interpreter to use.
- /usr/bin/env finds bash in your enviromment (more portable).

Make and Run the Script

1	chmod +x hello.sh	Make the script executable.
2	./hello.sh	Run it directly (uses the shebang).
3	bash hello.sh	Run it with Bash explicitly.

A Complete Example

```
$ cat hello.sh
#!/usr/bin/env bash
echo "Hello from my first script!"
exit 0
$ chmod +x hello.sh
$ ./hello.sh
Hello from my first script!
```

Exit Codes (Return Status)

Exit Code	Meaning
0	Success (command ran correctly).
1	General error or failure.
2	Misuse of a command or invalid input.
126	Found, but not executable (permission issue).
127	Command not found.

Basic Debugging Techniques

- Run with debug mode:
  `bash -x hello.sh`
- Check the last command's exit code:
  `echo $?`
- Catch errors in a script:
  `set -e  # stop on error`

How It Works (Execution Flow)



Script File (hello.sh) → Interpreter (Bash) → Execute Commands Line by Line → Return Exit Code (e.g., 0)

Key Takeaways

- A Bash script is a file with shell commands.
- Use #!/usr/bin/env bash at the top.
- Make it executable with chmod +x.
- Run it with ./script.sh or bash script.sh.
- Use exit codes and debug flags to find issues.

Junior Engineer Tip

Always test your script, check its exit code (echo $?), and use bash -x when things don't work as expected.

Variables, Quoting, and Environment Variables

What You'll Learn

- Work with variables in Bash.
- Understand single vs double quotes.
- Use command substitution.
- Export and use environment variables.
- Prevent word splitting and other common issues.

Variable Assignment and Expansion

- Create a variable: `NAME="VERIQTA"`
- Use the variable: `echo "$NAME"`
- Rules:
  - No spaces around '='.
  - Use letters, numbers, and _ (underscore).
  - Variables are case-sensitive (e.g., NAME ≠ name).

Single vs Double Quotes

Quoting	Behavior	Example & Output
Single quotes (')	Treats the text literally (no expansion).	`echo '$NAME'` Output: $NAME
Double quotes (")	Expands variables and commands.	`echo "$NAME"` Output: VERIQTA

Command Substitution

- Run a command and use its output in a variable or another command.
- Two common forms:

```
FILE_COUNT=$(ls -l | wc -l)
FILE_COUNT=`ls -l | wc -l`
```

- Example:

```
echo "Files: $FILE_COUNT"
```

Environment Variables

- Available to the current process and its child processes.
- Set and export a variable:

```
export APP_ENV=dev
echo "$APP_ENV"   # Output: dev
```

- Common examples:
  $HOME, $PATH, $USER, $PWD, $SHELL

Preventing Word Splitting

- Unquoted variables can be split into multiple words if they contain spaces.
- Bad (may split):

```
NAME= "Ann Ogechi"; echo $NAME
```

- Good (keeps it intact):

```
NAME="Ann Ogechi"; echo "$NAME"
```

Quick Example: Putting It Together

```
#!/usr/bin/env bash
APP="web"
COUNT=$(ls -1 | wc -l)
export APP
echo "App: $APP"
echo "Files: $COUNT"
```

→ Output:

App: web
Files: 5

Key Takeaways

- ✅ Use variables to store and reuse values.
- ✅ Single quotes (') keep text as-is.
- ✅ Double quotes (") expand variables.
- ✅ Use $(command) to capture command output.
- ✅ Use export to make a variable available to other processes.
- ✅ Always quote variables to prevent word splitting.

Reading Input and Processing Script Arguments

What You'll Learn

- Read input from the user with **read**.
- Work with script arguments (positional parameters).
- Use **$@**, **$#**, and **shift** correctly.
- Validate and safely handle user-supplied values.

Reading Input with read

- The **read** command gets input from the user and stores it in a variable.
- Example:

```
read -p "Enter your name: " NAME
echo "Hello, $NAME"
```

Positional Parameters (Script Arguments)

- Values passed to a script are stored as positional parameters.
- They are referenced as:

Parameter	Meaning
$0	The script name.
$1, $2, ...	The first, second, ... argument.
$#	The total number of arguments.
$@	All arguments (separate words).

Using $@ and $#

```
#!/usr/bin/env bash
echo "Script: $0"
echo "Total arguments: $#"
echo "All arguments: $@"
```

Run: `./info.sh devops linux bash`

Output:

```
Script: ./info.sh
Total arguments: 3
All arguments: devops linux bash
```

The shift Command

- **shift** moves the arguments to the left.
- It removes $1, so $2 becomes the new $1.
- Useful when processing multiple arguments.

```
echo "$1"    # First argument
shift
echo "$1"    # Now the second argument
```

Argument Validation

- Always check that the required arguments are provided before using them.

```
if [ $# -lt 2 ]; then
   echo "Usage: $0 <name> <age>"
   exit 1
fi
```

Safe Handling of User Input

- Quote variables to prevent word splitting and unexpected behavior.
- Example:

```
FILE="$1"
if [ -f "$FILE" ]; then
   echo "File exists: $FILE"
else
   echo "File not found: $FILE"
fi
```

Key Takeaways

- ✅ Use **read** to get input from the user.
- ✅ Use **$1, $2, ...** to access script arguments.
- ✅ Use **$#** to check how many arguments were provided.
- ✅ Use **$@** to work with all arguments.
- ✅ Use **shift** to process arguments one by one.
- ✅ Always validate and quote user-supplied values (e.g., **"$FILE"**).

Conditional Logic and Decision-Making

What You'll Learn

- Make decisions in scripts using **if, elif, else**.
- Use **test** and **[[ ]]** for conditions.
- Compare strings and numbers.
- Check file and directory conditions.
- Run commands based on conditions.

The if, elif, else Structure

```
if [ "$USER" = "root" ]; then
   echo "You are root."
elif [ "$USER" = "ubuntu" ]; then
   echo "Ubuntu user."
else
   echo "Regular user."
fi
```

test vs [[ ]]

Feature	test ([ ])	[[ ]]
Basic use	[condition]	[[condition]]
String compare	=, !=	=, !=, =~ (regex)
Logic (AND/OR)	-a, -o	&&, ||
Safer & modern	Works everywhere	Recommended in Bash

String Comparisons

Operator	Meaning	Example
=	Equal to	["$a" = "$b"]
!=	Not equal to	["$a" != "$b"]
-z	Empty string	[-z "$a"]
-n	Not empty	[-n "$a"]

Numeric Comparisons

Operator	Meaning	Example
-eq	Equal to	["$n" -eq 10]
-ne	Not equal to	["$n" -ne 10]
-gt	Greater than	["$n" -gt 10]
-ge	Greater or equal	["$n" -ge 10]
-lt	Less than	["$n" -lt 10]
-le	Less or equal	["$n" -le 10]

File Tests (Common)

Test	Meaning	Example
-e	File exists	[-e /etc/hosts]
-f	Is a regular file	[-f script.sh]
-d	Is a directory	[-d /var/log]
-r	Readable	[-r file.txt]
-w	Writable	[-w file.txt]
-x	Executable	[-x script.sh]

Conditional Execution (One-Liners)

- Run commands based on success or failure.

```
command && echo "Success"
command || echo "Failed"
```

- Example: Check if a file exists and act.

```
[ -f app.log ] && echo "Found" \
|| echo "Not found"
```

Key Takeaways

- ✅ Use **if, elif, else** to make decisions.
- ✅ **test ([ ])** works everywhere; **[[ ]]** is more powerful in Bash.
- ✅ Use **=** and **!=** for strings.
- ✅ Use **-eq, -gt, -lt,** etc. for numbers.
- ✅ Use **-e, -f, -d, -r, -w, -x** for file checks.
- ✅ Use **&&** and **||** for quick conditional execution.
- ✅ Always quote variables (e.g., **"$var"**).

Loops and Repetitive Automation

What You'll Learn

- Use for, while, and until loops.
- Control loops with break and continue.
- Automate repetitive tasks.
- Process files and command output safely.

Loop Overview

Loop	Purpose	Typical Use
for	Iterate over a list of values.	Files, words, or ranges.
while	Repeat while a condition is true.	Run until a condition changes.
until	Repeat while a condition is false.	Retry until success (e.g., service is up).

The for Loop

```
for item in devops linux bash; do
  echo "Topic: $item"
done
```

Output:
Topic: devops
Topic: linux
Topic: bash

The while Loop

```
count=1
while [ $count -le 3 ]; do
  echo "Count: $count"
  count=$((count + 1))
done
```

Output:
Count: 1
Count: 2
Count: 3

The until Loop

```
count=1
until [ $count -gt 3 ]; do
  echo "Count: $count"
  count=$((count + 1))
done
```

Output:
Count: 1
Count: 2
Count: 3

Loop Control: break and continue

- break: Exit the loop immediately.
- continue: Skip the current iteration and move to the next.

```
for n in {1..5}; do
  [[ $n -eq 3 ]] && continue
  [[ $n -eq 5 ]] && break
  echo "n: $n"
done
```

Output:
n: 1
n: 2
n: 4

Processing Files Safely

- Use loops to read and process files line by line.
- Quote variables to handle spaces and special characters.

```
while IFS= read -r line; do
  echo "Line: $line"
done < users.txt
```

users.txt
alice
bob
carol

Looping Through Command Output

- Use command substitution to loop over command results.

```
for file in $(ls *.log); do
  echo "Processing: $file"
  wc -l "$file"
done
```

Key Takeaways

- ✅ Use for for lists (e.g., files, words, ranges).
- ✅ Use while when a condition is true.
- ✅ Use until when a condition is false.
- ✅ Use break to exit a loop early.
- ✅ Use continue to skip to the next iteration.
- ✅ Quote variables ("$var") to handle spaces.
- ✅ Use read -r and IFS to safely process file content.

Functions, Scope, and Reusable Bash Code

What You'll Learn

- Create and use functions in Bash.
- Pass parameters to functions.
- Use local variables and understand scope.
- Return statuses and handle output.
- Design reusable and maintainable script components.

Function Declarations

- A function groups commands and can be reused.
- Two common forms:

```
name() {
  commands
}
```

```
function name {
  commands
}
```

A Simple Function

```
greet() {
  echo "Hello, $1!"
}
greet "DevOps"
```

Output:
Hello, DevOps!

Function Parameters

- Use $1, $2, ... to access arguments passed to a function.
- Example:

```
sum() {
  echo $(( $1 + $2 ))
}
sum 5 7
```

Output:
12

Variable Scope (Global vs Local)

- Variables are global by default (accessible everywhere).
- Use local to keep a variable inside a function.

```
show() {
  local msg="Inside function"
  echo "$msg"
}
show
echo "$msg"    # empty (not global)
```

Return Status (Exit Code)

- Functions don't return values; they return an exit status (0 = success, non-zero = failure).
- Use return to set the status and check it with $?.

```
check_file() {
  [ -f "$1" ] && return 0 || return 1
}
check_file "/etc/hosts"
if [ $? -eq 0 ]; then
  echo "File exists"
else
  echo "File not found"
fi
```

Output Handling

- Use echo or printf to return data from a function.
- You can capture the output in a variable using command substitution.

```
get_date() {
  date +"%Y-%m-%d"
}
today=$(get_date)
echo "Today: $today"
```

Output:
Today: 2026-11-12

Best Practices for Reusable Functions

- ✅ Use clear, descriptive function names.
- ✅ Keep functions focused on one task.
- ✅ Use local for intenal variables.
- ✅ Validate inputs (e.g., check for empty values).
- ✅ Use meaningful exit codes (0 = success, >0 = error).
- ✅ Organize related functions together (e.g., in a library file).

Arrays and Structured Data Processing

What You'll Learn

- Work with indexed and associative arrays in Bash.
- Access and modify array elements.
- Expand and iterate over arrays.
- Handle array data safely.
- Use arrays for real infrastructure tasks (e.g., host inventory).

Array Types in Bash

Type	Syntax Example	Use Case
Indexed Array	servers=("web1" "web2")	Ordered list (uses numbers 0, 1, 2, ...).
Associative Array	declare -A srv srv[web1]="10.0.0.10"	Key-value pairs (e.g., name → IP).

Indexed Arrays

```
servers=("web1" "web2" "db1")
echo "${servers[0]}"    # web1
echo "${#servers[@]}"   # 3
```

- Create: arr=(a b c)
- Access: ${arr[0]} (first element)
- Length: ${#arr[@]} (total elements)

Associative Arrays (Key-Value)

```
declare -A hosts
hosts[web1]="10.0.0.10"
hosts[web2]="10.0.0.11"
echo "${hosts[web1]}"    # 10.0.0.10
```

- Use declare -A to create.
- Access by key: ${hosts[web1]}
- List keys: ${!hosts[@]}

Array Expansion and Iteration

```
# Iterate over an array
for srv in "${servers[@]}"; do
  echo "Checking: $srv"
done
```

- "${array[@]}" → each element (safe).
- ${array[*]} → all elements as one string.
- Use quotes to preserve spaces in values.

Safe Element Handling

- Always quote array expansions: "${arr[@]}".
- Check if an array is empty:

```
if [ ${#arr[@]} -eq 0]; then echo "Empty"; fi
```

- Use defaults to avoid unset errors:

```
echo "${arr[0]:-N/A}"   # Use N/A if not set
```

- Validate data before use (e.g., non-empty, IP format, etc.).

Practical Example: Infrastructure Inventory

```
#!/usr/bin/env bash
declare -A inventory
inventory[web]="10.0.0.10"
inventory[app]="10.0.0.20"
inventory[db]="10.0.0.30"
echo "Server Inventory"
echo "---------------"
for role in "${!inventory[@]}"; do
  printf "%-5s -> %s\n"
    "$role" "${inventory[$role]}"
  done
```

Output:

```
Server Inventory
---------------
web  -> 10.0.0.10
app  -> 10.0.0.20
db   -> 10.0.0.30
```

Key Takeaways

- Bash supports indexed and associative arrays.
- Use "${array[@]}" for safe iteration.
- Get the array size with ${#array[@]}.
- Use declare -A for key-value data.
- Always quote expansions to avoid word splitting.
- Arrays are powerful for real tasks (e.g., server lists, config maps, inventory).

Standard Input, Output, Errors, and Redirection

What You'll Learn

- Understand standard input, output, and error streams.
- Use file descriptors (0, 1, 2).
- Redirect and append output to files.
- Combine and capture command output.
- Use pipes to build powerful workflows.

Standard Streams and File Descriptors

FD	Stream	Meaning	Default
0	stdin	Standard input	Keyboard (input)
1	stdout	Standard output	Terminal (output)
2	stderr	Standard error	Terminal (errors)

How It Works



Common Redirection Operators

Operator	Meaning	Example
>	Redirect (overwrite)	ls > files.txt
>>	Redirect (append)	echo "log" >> app.log
2>	Redirect stderr	cmd 2> error.log
2>&1	Combine stderr with stdout	cmd > all.log 2>&1

Pipes (|)

- Send the output of one command to another command.
- Useful for filtering, searching, and processing data.

```
ps aux | grep nginx
cat access.log | wc -l
```

Capturing Command Output

- Store command output in a variable using $().
- Use it for logic, reporting, or further processing.

```
COUNT=$(ls | wc -l)
echo "Total files: $COUNT"
IP=$(hostname -I | awk '{print $1}')
echo "Server IP: $IP"
```

Examples in Practice

- Redirect output to a file:

```
df -h > disk_usage.txt
```

- Append output to a log:

```
echo "Backup $(date)" >> backup.log
```

- Capture both output and errors:

```
./deploy.sh > deploy.log 2>&1
```

- Use a pipe to filter results:

```
systemctl list-units | grep running
```

Key Takeaways

- ✅ Linux uses three standard streams: stdin (0), stdout (1), stderr (2).
- ✅ Use > to write (overwrite) and >> to append.
- ✅ Use 2> to capture errors.
- ✅ Use 2>&1 to combine errors with output.
- ✅ Use | to pipe output between commands.
- ✅ Use $() to capture command output in a variable.
- ✅ These tools help you automate, log, and troubleshoot scripts reliably.

Text Processing with grep, sed, awk, and jq

What You'll Learn

- Search and filter text with grep.
- Transform and edit text with sed.
- Extract and process fields with awk.
- Query and filter JSON data with jq.
- Choose the right tool for structured vs unstructured data.

Tool Overview

Tool	Purpose	Best For
grep	Search and filter text using patterns.	Logs, config files, plain text.
sed	Stream editor for find/replace and edits.	Text transformation and cleanup.
awk	Process text by fields and patterns.	Column extraction, reports, quick analysis.
jq	Query and filter JSON data.	Structured JSON (APIs, logs, configs).

grep: Search and Filter Text

```
grep -i "error" app.log
grep -E "error|warn" app.log
grep -v "health" app.log
```

- -i : case-insensitive search
- -E : use extended patterns (|, +, ?)
- -v exclude matching lines

sed: Transform and Edit Text

```
sed 's/http/https/g' config.txt
sed -n '1,10p' app.log
sed '/DEBUG/d' app.log
```

- s/old/new/g : replace text (global)
- -n '1,10p' : print lines 1 to 10
- /PATTERN/d : delete matching lines

awk: Extract and Process Fields

```
awk '{print $1, $3}' access.log
awk -F, '{print $2}' users.csv
awk '$NF ~ /ERROR/ {print $0}' app.log
```

- $1, $2, ... : fields (columns)
- -F, : set field separator (e.g., comma)
- $NF : last field
- ~ : pattern match (regex)

jq: Query and Filter JSON

```
jq '.name' server.json
jq '.users[] | .name' users.json
jq '.services[] | select(.status=="up") |
.name' status.json
```

- .name : get a field
- [] : iterate over an array
- select() : filter by condition
- Use for structured JSON (APIs, cloud, configs).

Choosing the Right Tool

Use Case	Recommended Tool
Find lines in logs (e.g., ERROR)	grep
Replace or clean up text	sed
Extract and summarize columns (e.g., logs, CSV)	awk
Query structured JSON (e.g., API or config output)	jq

Practical Example: Log Analysis

```
grep "ERROR" app.log |
awk '{print $1, $4}' |
sed 's/://g' |
sort | uniq -c | sort -nr
```

Output:

```
15 ERROR
 7 TIMEOUT
 3 FAILED
```

- Finds ERROR lines, extracts fields, cleans ":", counts unique errors, and sorts by frequency.

Key Takeaways

- ✅ Use grep to search and filter text quickly.
- ✅ Use sed to transform or edit text.
- ✅ Use awk to extract, format, and summarize data.
- ✅ Use jq for JSON and other structured data.
- ✅ Use a pipeline (|) to combine tools for powerful tasks.
- ✅ Choose the tool based on the data type: unstructured (text/logs) vs structured (JSON, CSV, API output).

File Handling, Paths, and Filesystem Automation

What You'll Learn

- Work with absolute and relative paths.
- Navigate directories efficiently.
- Create and manage files safely.
- Use temporary files and cleanup.
- Create and use symbolic links.
- Avoid destructive filesystem actions.

Paths: Absolute vs Relative

Path Type	Example	Meaning
Absolute	/var/log/app.log	Full path from root (/).
Relative	logs/app.log	Path relative to current directory.
Home	~/projects	Your home directory.
Parent	../scripts	One level up.
Current	./run.sh	Current directory.

Directory Traversal (Navigation)

```
pwd                 # show current path
cd ~/projects       # go to a directory
cd ..               # go up one level
cd -                # go to previous dir
ls -la              # list files (details)
```

Safe File Creation and Management

```
mkdir -p /tmp/work/reports    # create dirs
touch report.txt              # create a file
cp report.txt report.bak      # copy file
mv report.bak archive/        # move/rename
rm -i old.log           # confirm before delete
```

Temporary Files and Cleanup

```
tmpfile=$(mktemp /tmp/app.XXXXXX)
echo "Processing..." > "$tmpfile"
# ... use the temp file ...
rm -f "$tmpfile"          # cleanup
```

- mktemp creates a unique, safe temporary file.
- Always clean up temporary files when done.

Symbolic Links

```
ln -s /var/www/app /opt/app   # create symlink
ls -l /opt/app                # shows -> target
readlink -f /opt/app          # resolve full path
```

- ln -s creates a symbolic (soft) link.
- Use links for flexible paths (e.g., deployments).
- readlink -f shows the actual (target) path.

Filesystem Automation Example

```
#!/usr/bin/env bash
BASE="/var/backups"
DATE=$(date +%F)
DEST="$BASE/$DATE"
mkdir -p "$DEST"
tar -czf "$DEST/app.tar.gz" /opt/app
echo "Backup saved: $DEST/app.tar.gz"
```

Output:

```
Backup saved:
/var/backups/
2026-11-12/
app.tar.gz
```

Avoiding Destructive Operations

- Use -i (interactive) or -n (dry run) with rm.
- Always verify paths before deleting.
- Avoid rm -rf / or untested wildcards (*).
- Test scripts in a non-production directory.
- Use find with care (e.g., find /path -type f ...).
- Keep backups or use version control (e.g., Git) for important files.

Key Takeaways

- Use absolute paths for scripts and automation.
- Prefer mkdir -p, touch, and mktemp for safe file work.
- Use symbolic links for flexibility and easier management.
- Always clean up temporary files.
- Be cautious with rm; validate paths and test first.
- Automate routine filesystem tasks (e.g., backups, archiving, log rotation) safely.

Error Handling and Defensive Bash Scripting

What You'll Learn

- Use exit statuses to detect failures.
- Write safer scripts with set -euo pipefail.
- Understand strict mode limitations.
- Perform explicit error checks.
- Use traps for cleanup and failure handling.
- Make scripts fail predictably and safely.

Exit Statuses (Return Codes)

Code	Meaning	Example
0	Success (no error)	command && echo "OK"
1	General error	test -f file || echo "Error"
126	Not executable	./script.sh # no execute perm
127	Command not found	badcmd # command missing

Strict Mode: set -euo pipefail

```
set -euo pipefail
# -e : exit on error
# -u : treat unset variables as error
# -o pipefail : fail if any command in
# a pipeline fails
```

- Helps catch errors early and avoid silent failures.
- Useful for reliable automation and CI scripts.

Limitations of Strict Mode

- It can't catch all logical errors.
- Some commands (e.g., grep, test) may return non-zero by design.
- Not a replacement for proper error handling.
- Use it as a safety net, not a complete solution.

Example:

```
grep "pattern" file.txt || true
# avoid exit when no match is found
```

Explicit Error Checks

```
if ! cp source.txt dest.txt; then
    echo "Copy failed!" >&2
    exit 1
fi
```

- Check command success with if, !, or $?.
- Log meaningful error messages (to stderr).
- Use clear exit codes for predictable failures.

Traps and Cleanup Handlers

```
cleanup() {
    echo "Cleaning up..."
    rm -rf "$TMPDIR"
}
trap cleanup EXIT ERR INT
```

- trap runs a function on script exit, error, or interrupt.
- Use it to clean up temporary files, locks, or resources.
- Common signals: EXIT (normal), ERR (error), INT (Ctrl+C).

Practical Example: Defensive Script

```
#!/usr/bin/env bash
set -euo pipefail
TMPDIR=$(mktemp -d)
cleanup() { rm -rf "$TMPDIR"; }
trap cleanup EXIT ERR INT
echo "Processing..."
cp /etc/hosts "$TMPDIR/hosts"
echo "Success: file copied."
```

Output:

```
Processing...
Success: file copied.
```

Key Takeaways

- ✅ Check and use exit statuses (0 = success, non-zero = error).
- ✅ Use set -euo pipefail for stricter, safer scripts.
- ✅ Know the limits of strict mode and handle expected non-zero cases.
- ✅ Perform explicit checks with if and $?.
- ✅ Use trap for reliable cleanup and graceful failure.
- ✅ Write predictable, defensive scripts that fail fast, log clearly, and leave no mess.

Debugging, Testing, and Static Analysis

What You'll Learn

- Find and fix issues in Bash scripts.
- Use bash -n and bash -x effectively.
- Analyze scripts with ShellCheck.
- Write automated tests with Bats.
- Use test fixtures and mock external commands.
- Investigate and resolve unexpected script behavior.

Key Debugging Tools

Tool	Purpose	Example
bash -n	Check syntax (no run)	bash -n script.sh
bash -x	Trace execution (commands and values)	bash -x script.sh
ShellCheck	Static analysis and best-practice checks	shellcheck script.sh
Bats	Automated testing framework for Bash	bats tests/

Syntax Check: bash -n

```
bash -n deploy.sh    # check syntax only
echo $?              # 0 = OK, non-zero = error
```

- Validates script syntax without executing it.
- Use it early (e.g., in CI) to catch errors fast.

Execution Trace: bash -x

```
bash -x deploy.sh        # trace execution
bash -xv deploy.sh       # also show script lines
```

- Shows each command and its expanded values.
- Useful for understanding script flow and failures.

ShellCheck: Static Analysis

```
shellcheck deploy.sh
```

- Finds common bugs and bad practices (e.g., quoting, unused variables).
- Follow the recommendations to write safer, cleaner, and more portable scripts.

Bats: Testing Bash Scripts

```
tests/
├── test_deploy.bats     # Bats test file
├── fixtures/            # test data (e.g., logs)
└── mocks/               # mock commands
```

- Bats provides a simple, structured way to test Bash.
- Organize tests, fixtures (sample data), and mocks (fake commands) for reliable and repeatable tests.

Example: Write and Run a Bats Test

```
# tests/test_backup.bats
@test "creates backup file" {
  run ./backup.sh /tmp/data
  [ "$status" -eq 0 ]
  [ -f "/tmp/data.bak" ]
}
```

Output:

✓ creates backup file
1 test, 0 failures

Mocking External Commands

```
# tests/mocks/aws
#!/usr/bin/env bash
echo '{"Status":"OK"}'
chmod +x tests/mocks/aws
PATH="tests/mocks:$PATH" bats tests/
```

- Replace real external commands (e.g., aws, curl) with simple mocks that return controlled output.
- Keeps tests fast, safe, and independent of real services.

Investigating Unexpected Behavior

- ☑ Reproduce the issue with a minimal example.
- ☑ Add temporary debug output (echo/printf).
- ☑ Use bash -x to trace the failing path.
- ☑ Check variable values, quoting, and exit statuses.
- ☑ Verify external dependencies and their outputs.

Key Takeaways

- ☑ Use bash -n (syntax) and bash -x (trace) to debug fast.
- ☑ Run ShellCheck to catch issues and follow best practices.
- ☑ Use Bats for automated, repeatable testing.
- ☑ Use fixtures and mocks to test safely and predictably.
- ☑ Investigate failures systematically and fix the root cause.

Process Management, Signals, and Job Control

What You'll Learn

- Work with process IDs (PIDs) and jobs.
- Run and manage background jobs.
- Use wait to coordinate processes.
- Send and handle signals (e.g., SIGTERM).
- Set timeouts for long-running commands.
- Manage subprocesses safely.
- Avoid orphaned or runaway processes.

Core Commands for Process Management

Command	Purpose	Example
ps	View running processes	ps aux | grep nginx
jobs	List shell jobs	jobs -l
&	Run a command in background	backup.sh &
wait	Wait for a job or PID	wait $PID
kill	Send a signal to a process	kill -TERM 1234
pkill	Kill by process name	pkill -f app.sh
timeout	Run a command with a time limit	timeout 60s ./deploy.sh

Process IDs (PIDs)

```
ps -e | grep app.sh
PID=$!        # PID of last background job
echo "App PID: $PID"
```

- Every process has a unique Process ID (PID).
- Use $! to get the PID of the last background job.

Job Control (Foreground and Background)

```
./backup.sh &        # run in background
jobs                 # list background jobs
fg %1                # bring job to foreground
bg %1                # send job to background
```

Signals and Process Control

Signal	Number	Meaning	Example
SIGTERM	15	Graceful stop (default)	kill -TERM $PID
SIGKILL	9	Force stop (cannot ignore)	kill -9 $PID
SIGHUP	1	Reload/restart process	kill -HUP $PID
SIGINT	2	Interrupt (Ctrl+C)	kill -INT $PID
SIGUSR1	10	User-defined (custom use)	kill -USR1 $PID

Timeout and Safe Execution

```
timeout 30s long_task.sh
if [ $? -eq 124 ]; then
    echo "Task timed out"
fi
```

- Use timeout to limit how long a command runs.
- Exit code 124 means the command timed out.

Managing Subprocesses

```
# run a subprocess and capture its PID
( ./worker.sh ) &
PID=$!
wait $PID          # wait for it to finish
```

- Use subshells (...) to group commands.
- Use wait to make your script reliably handle subprocesses.

Avoiding Orphaned Processes

```
cleanup() {
    echo "Stopping child processes..."
    pkill -P $$ 2>/dev/null || true
}
trap cleanup EXIT INT TERM
```

- Use trap to clean up child processes on exit or signals.
- pkill -P $$ stops processes started by this script.
- Prevents orphaned or runaway processes.

Example: Safe Background Job with Cleanup

```
#!/usr/bin/env bash
cleanup() {
echo "Cleaning up..."; kill -TERM $PID 2>/dev/null; }
trap cleanup EXIT INT TERM
./server.sh &
PID=$!
wait $PID
```

Output:

Cleaning up...
(on exit or Ctrl+C)

Key Takeaways

- ✅ Use PIDs and job control to manage processes.
- ✅ Use wait to coordinate and avoid race conditions.
- ✅ Send the right signals (e.g., SIGTERM vs SIGKILL).
- ✅ Use timeout to prevent long-running tasks.
- ✅ Use trap for cleanup on exit or interruption.
- ✅ Always clean up child processes to avoid orphans.

Scheduling Automation with cron and systemd Timers

What You'll Learn

- Automate tasks with cron and systemd timers.
- Write and understand cron expressions.
- Create and manage systemd timer units.
- Understand execution environments and logging.
- Prevent overlapping jobs with locking.
- Recover and handle failed scheduled tasks.

Cron Expression Quick Reference

Field	Values	Meaning
Minute	0–59	Minute of the hour
Hour	0–23	Hour of the day
Day of Month	1–31	Day of the month
Month	1–12 or JAN–DEC	Month of the year
Day of Week	0–7 or SUN–SAT	Day of the week (0 or 7 = Sunday)

Cron: Examples and Usage

```
# Edit your cron jobs
crontab -e
# Run at 2:30 AM daily
30 2 * * * /opt/scripts/backup.sh
# Run every 15 minutes
*/15 * * * * /opt/scripts/health.sh
```

- Use crontab -e to add or edit jobs.
- Use standard 5-field cron expressions.
- Set absolute paths in scheduled scripts.

systemd Timer: Units and Example

backup.service

```
[Unit]
Description=Backup Job

[Service]
Type=oneshot
ExecStart=/opt/scripts/backup.sh
```

backup.timer

```
[Unit]
Description=Run Backup Daily

[Timer]
OnCalendar=*-*-* 02:30:00
Persistent=true

[Install]
WantedBy=timers.target
```

- A .service defines what to run (the job).
- A .timer defines when to run it (the schedule).
- Use Persistent=true to run missed jobs after downtime.

Execution Environment and Logging

```
# Use full paths and control the environment
PATH=/usr/local/bin:/usr/bin:/bin \
    /opt/scripts/backup.sh \
    >> /var/log/backup.log 2>&1
```

- Cron and systemd run with a minimal environment.
- Always use absolute paths for commands and files.
- Log output to a file (e.g., /var/log/...) for troubleshooting.

Preventing Overlapping Jobs (Locking)

```
#!/usr/bin/env bash
LOCK=/var/lock/backup.lock
if ! flock -n "$LOCK" -c "/opt/scripts/backup.sh"; then
  echo "Another backup is already running."
  exit 0
fi
```

- Use flock to ensure only one instance runs at a time.
- Prevents overlapping and resource conflicts.

Monitoring and Recovery

```
systemctl list-timers --all      # view timers
systemctl status backup.timer    # check status
journalctl -u backup.service     # view logs
systemctl start backup.timer     # re-run if needed
```

- Use systemctl to view and manage timers.
- Use journalctl to check logs and failures.
- Re-run failed jobs manually or adjust the schedule.

Key Takeaways

- ✓ Use cron for simple, user-level scheduling.
- ✓ Use systemd timers for more reliable, production-grade scheduling.
- ✓ Always set absolute paths and handle a minimal environment.
- ✓ Log job output and errors for visibility.
- ✓ Prevent overlapping jobs with file locking (flock).
- ✓ Use Persistent=true and monitoring for job recovery.

Secure Shell Scripting for Production Systems

What You'll Learn

- Write secure and production-ready Bash scripts.
- Apply the principle of least privilege.
- Use sudo safely and intentionally.
- Handle credentials and secrets securely.
- Prevent command injection and related risks.
- Work with secure temporary files.
- Execute commands safely on remote systems.

Least Privilege and Safe sudo

Practice	Example
Least privilege	Run tasks with a dedicated, limited user
Safe sudo	sudo -u appuser /opt/app/deploy.sh
Limit sudo scope	Use visudo to allow only specific commands
Avoid sudo -i	Don't run scripts with a full root shell
Log and audit	Ensure sudo actions are logged

Credential Handling and Secrets

```
# Load secrets from a protected file
set -o allexport
source /etc/app/.env  # chmod 600 .env
set +o allexport
DB_PASS="${DB_PASS:?Missing DB_PASS}"
```

- Keep secrets in files with strict permissions (e.g., chmod 600).
- Use environment variables, not hardcoded secrets.
- Validate that required secrets are set.

Preventing Command Injection

```
# Bad (vulnerable)
cmd="ls $user_input"        # risk: injection

# Good (safe)
cmd=(ls -- "$user_input")   # use an array
"${cmd[@]}"                 # preserves input
```

- Never build commands from untrusted input.
- Use arrays and proper quoting ("...").
- Validate and sanitize all external input.

Secure Temporary Files

```
# Create a secure temporary file
tmpfile=$(mktemp)
trap 'rm -f "$tmpfile"' EXIT
echo "Processing..." > "$tmpfile"
```

- Use mktemp to create unique, secure files.
- Set a trap to clean up temporary files.
- Avoid predictable filenames (e.g., /tmp/file.txt).

Secret Exposure Risks to Avoid

Risk	Safer Approach
Hardcoded secrets	Use env vars or a secrets manager
Secrets in logs	Mask or avoid logging sensitive data
World-readable files	Use chmod 600 for secret files
Process arguments	Avoid passing secrets on the command line
Git commits	Never commit secrets to version control

Safe Remote Execution (SSH)

```
# Run a command safely on a remote host
ssh -o StrictHostKeyChecking=yes \
app@server01 "bash -s" <<'EOF'
   set -euo pipefail
   cd /opt/app
   ./deploy.sh
EOF
```

Key Points:

- ✓ Use SSH keys (not passwords).
- ✓ Verify the host (StrictHostKeyChecking).
- ✓ Pass and run a minimal, controlled script.

Key Takeaways

- ✓ Run scripts with the least privilege required.
- ✓ Use sudo safely and limit what it can do.
- ✓ Handle credentials and secrets securely.
- ✓ Validate input and prevent command injection.
- ✓ Create and clean up secure temporary files.
- ✓ Avoid exposing secrets in code, logs, or processes.
- ✓ Use SSH with keys and host verification for remote execution.

Automating Linux Server Administration

What You'll Learn

- Check service health and restart on failure.
- Monitor disk usage and inode usage.
- Inspect and rotate logs.
- Validate system configuration files.
- Check installed packages and updates.
- Automate safe maintenance tasks.

Service Health Checks

Service	Command
nginx	`systemctl is-active nginx`
ssh	`systemctl is-active ssh`
mysql	`systemctl is-active mysql`
systemd	`systemctl status <service>`
Check & restart	`if ! systemctl is-active --quiet nginx; then systemctl restart nginx; fi`

Disk Monitoring

```
df -h | awk 'NR==1 || $5+0 > 80 {print}'
df -i | awk 'NR==1 || $5+0 > 80 {print}'
```

- Check disk usage and inode usage.
- Alert when usage is high (e.g., > 80%).
- Use logrotate for log file management.

Log Inspection

```
tail -n 50 /var/log/syslog
grep -i error /var/log/nginx/error.log
journalctl -u nginx --since "1 hour ago"
```

- Check for errors, warnings, and failed attempts.
- Use grep, awk, and journalctl for analysis.
- Rotate and clean old logs.

Configuration Validation

```
nginx -t
sshd -t
test -f /etc/myapp/config.yml &&
    echo "OK" || echo "Missing"
```

- Validate config files before applying changes.
- Check syntax and required files.
- Fail fast if validation fails.

Package Checks

```
dpkg -l | grep -E 'nginx|mysql'
rpm -qa | grep -E 'httpd|mysql'
apt list --upgradable
```

- Verify required packages are installed.
- Check for available updates.
- Automate with apt or dnf as needed.

Safe Maintenance Automation

```
#!/bin/bash
set -euo pipefail
apt update && apt upgrade -y
systemctl restart nginx
echo "Maintenance completed: $(date)"
```

- Use safe flags (set -euo pipefail).
- Log all actions and results.
- Run maintenance during low-traffic hours.

Key Best Practices

- [x] Run health checks and monitoring automatically.
- [x] Use safe automation with error handling.
- [x] Validate configs and services before changes.
- [x] Keep logs, rotate them, and review regularly.
- [x] Check packages and apply updates safely.
- [x] Use systemd timers or cron for scheduling.
- [x] Avoid hardcoded values and sensitive data.
- [x] Log everything and alert on failures.

Shell Scripting for DevOps, Cloud, and CI/CD

What You'll Learn

- Use Bash to automate DevOps tasks.
- Script Git, Docker, Kubernetes, and AWS CLI operations.
- Build reusable deployment and verification scripts.
- Automate CI/CD pipeline steps.
- Handle errors and produce useful logs.

Git Automation (Version Control)

```
#!/bin/bash
set -euo pipefail
git pull origin main
git add . && git commit -m "Auto: update"
git push origin main
```

- Automate common Git workflows (pull, commit, push).
- Use set -euo pipefail for reliable execution.

Docker Automation

```
docker build -t myapp:$(date +%Y%m%d) .
docker tag myapp:latest myapp:v1.0.0
docker push myapp:v1.0.0
```

- Build, tag, and push Docker images with scripts.
- Automate image cleanup and disk space checks.
- Use clear tags (e.g., version or build date).

Kubernetes Diagnostics

```
kubectl get pods -A
kubectl describe pod <pod-name> -n <ns>
kubectl logs <pod-name> -n <ns> --tail=200
kubectl rollout status deploy/<app> -n <ns>
```

- Check pod status, events, and logs automatically.
- Monitor deployments and rollout progress.
- Collect cluster diagnostics for troubleshooting.

AWS CLI Automation

```
aws s3 sync ./build s3://myapp-artifacts/
aws ec2 describe-instances \
  --query 'Reservations[].Instances[].State.Name'
aws sts get-caller-identity
```

- Automate common AWS tasks (S3, EC2, IAM).
- Use --query to get clean, useful output.
- Verify the correct AWS identity and permissions.

CI/CD Pipeline Scripting

```
#!/bin/bash
set -e
./run-tests.sh
docker build -t myapp:$CI_COMMIT_SHA .
docker push myapp:$CI_COMMIT_SHA
```

- Script the build, test, and image push steps.
- Use CI variables (e.g., $CI_COMMIT_SHA) for tagging.
- Fail fast when a step returns a non-zero exit code.

Deployment Verification

```
#!/bin/bash
set -e
URL="https://myapp.example.com/health"
if curl -fsS "$URL" >/dev/null; then
  echo "✅ Deployment successful"
else
  echo "❌ Health check failed"
  exit 1
fi
```

Checks to Automate:

- ✓ Service is running and healthy.
- ✓ Correct version is deployed.
- ✓ Endpoints return the expected status.
- ✓ Rollback if the check fails.

Key Takeaways

- ✓ Use Bash to automate end-to-end DevOps tasks.
- ✓ Work with Git, Docker, Kubernetes, and AWS CLI.
- ✓ Build CI/CD pipeline scripts (build, test, push, deploy).
- ✓ Verify deployments with health checks and logs.
- ✓ Use strict mode (set -euo pipefail) and clear logs.
- ✓ Make scripts reusable, parameterized, and secure.

Production-Grade Shell Scripting Architecture

What You'll Learn

- Design robust, production-grade Bash scripts.
- Build idempotent and safe automation.
- Add retries, timeouts, and backoff.
- Control concurrency and avoid conflicts.
- Implement structured logging and config management.
- Support dry-run and rollback operations.

Key Architectural Principles

Principle	Description
Idempotency	Safe to run multiple times with the same result.
Retries & Backoff	Automatically retry failed operations with increasing delay.
Timeouts	Prevent long-running or hanging tasks.
Concurrency Control	Limit parallel runs and avoid conflicts.
Structured Logging	Consistent, machine-readable logs (e.g., JSON).
Rollback	Revert changes when a step fails.

Idempotency: Safe and Repeatable

```bash
if [ -f "/opt/app/installed" ]; then
  echo "Already installed, skipping..."
else
  install_app && touch /opt/app/installed
fi
```

- Check state before making changes.
- Use a marker (file, tag, or state) to track completion.

Retries with Exponential Backoff

```bash
retry() {
  local n=0 max=5 delay=2
  until "$@"; do
    n=$((n+1)); [ $n -ge $max ] && return 1
    sleep $((delay ** n))  # 2, 4, 8, 16...
  done
}
```

- Retry transient failures (e.g., network, API, package).
- Use exponential backoff to reduce system load.

Timeouts and Concurrency Control

```bash
# Timeout a long-running command (60 seconds)
timeout 60 ./deploy.sh
# Prevent concurrent runs with a lock
exec 9>/var/lock/deploy.lock
flock -n 9 || { echo "Another run in progress"; exit 1; }
```

- Use timeout to avoid hanging tasks.
- Use flock (or lock files) to prevent overlapping runs.

Structured Logging

```bash
LOG_FILE="/var/log/app-deploy.log"
log () {
  echo "$(date -Iseconds) | $1 | $2" >> "$LOG_FILE"
}
log "INFO" "Starting deployment..."
```

- Use a consistent log format (e.g., ISO timestamp, level).
- Log key steps, results, and errors for easy troubleshooting.

Configuration, Dry-Run, and Rollback

```bash
# Read config and support dry-run
source /etc/myapp/config.env
DRY_RUN=false
[[ "${1:-}" == "--dry-run" ]] && DRY_RUN=true

run_or_echo() { if $DRY_RUN; then echo "[DRY-RUN] $*";
 else eval "$@"; fi; }
```

- Use a config file (e.g., .env or /etc/app/config.env).
- Add --dry-run to preview actions without making changes.
- Implement rollback to restore the previous state on failure.

Key Takeaways

- [x] Make scripts idempotent and safe to re-run.
- [x] Add retries with exponential backoff for resilience.
- [x] Use timeouts to prevent stuck or long-running jobs.
- [x] Control concurrency to avoid conflicts and resource issues.
- [x] Implement structured, consistent logging.
- [x] Load configuration from external files or environment vars.
- [x] Support a dry-run mode for safe testing.
- [x] Design a rollback strategy for predictable recovery.

Final Challenge: Build a Production-Ready DevOps Automation Toolkit

Project Objective

- Develop a modular Bash toolkit that automates server health checks, validates dependencies, collects diagnostics, logs execution, handles failures, supports safe configuration, and includes automated tests.

Requirements

- Production-ready, modular, and reusable.
- Must perform health checks (CPU, memory, disk, services).
- Validate required tools and dependencies.
- Collect and package diagnostics (logs, system, services).
- Use structured logging and proper error handling.
- Support a config file and a safe dry-run mode.
- Include automated tests (e.g., Bats) for key functions.

Toolkit Structure (Example)

```
devops-toolkit/
├── bin/          # main executable
├── lib/          # reusable functions
├── modules/      # health, deps, diag
├── config/       # config files (YAML/ENV)
├── logs/         # execution logs
├── tests/        # automated tests (Bats)
└── README.md     # usage and setup
```

Core Modules

Module	Purpose
health.sh	Check CPU, memory, disk, and service health.
deps.sh	Validate required tools (git, docker, kubectl, etc.).
diagnostics.sh	Collect system, service, and application logs.
log.sh	Provide structured, timestamped logging.
config.sh	Load and validate configuration (ENV/YAML).
utils.sh	Common functions (retry, timeout, cleanup, etc.).

Key Features

- ✅ Comprehensive server health checks.
- ✅ Dependency and version validation.
- ✅ Diagnostics collection and log packaging.
- ✅ Structured logging (levels: INFO, WARN, ERROR).
- ✅ Robust error handling with retries and timeouts.
- ✅ Config-driven behaviour (e.g., config.yml or .env).
- ✅ Dry-run mode to preview actions safely.
- ✅ Automated tests using Bats.

Example Usage

```
# Make executable
chmod +x bin/devops-toolkit
# Run a full check
./bin/devops-toolkit check
# Collect diagnostics
./bin/devops-toolkit diag --output ./reports
# Dry run with a custom config
./bin/devops-toolkit check --dry-run --config config.yml
```

Automated Testing (Bats)

```
# tests/health.bats
@test "health check runs successfully" {
  run ./bin/devops-toolkit check
  [ "$status" -eq 0 ]
  [[ "$output" == *"Health Check Summary"* ]]
}
```

- Write tests for health checks, dependency checks, and configuration loading.
- Ensure failures are handled and reported correctly.

Deliverables & Success Criteria

- ✅ A complete, well-structured Bash toolkit (GitHub repo).
- ✅ Modular scripts with clear function separation.
- ✅ Config file with example settings.
- ✅ Logs generated in a consistent, structured format.
- ✅ Automated tests with passing results.
- ✅ A README with setup, usage, and example outputs.
- ✅ Demonstration: run the toolkit on a Linux server/VM and show a successful health report and diagnostics output.